
Adversarially Trained Restricted Boltzmann Machines

Andrew Yang

Department of Computer Science
Brown University
Providence, RI 02912
ayang103@cs.brown.edu

Jason Eveleth

Department of Computer Science
Brown University
Providence, RI 02912
jeveleth@cs.brown.edu

Abstract

The restricted Boltzmann machine (RBM) is a type of stochastic neural network designed for unsupervised learning. We focus on the Gaussian-Bernoulli RBM, an RBM variant that learns the distribution of continuous-valued, high-dimensional data. Gaussian-Bernoulli RBMs are notoriously difficult to train; over the past decade, many new algorithms have been designed to improve the robustness of their training. In this work, we provide an overview of RBMs and the contrastive divergence training algorithm. Then, we investigate a previously proposed adversarial training algorithm which builds upon the simplest formulation of contrastive divergence learning. Finally, we extend the adversarial training approach to conditional RBMs. We show—through experiments on Gaussian mixtures and MNIST—that despite their two-layered architecture, RBMs can learn high dimensional distributions well.

1 Background

1.1 Motivation

Gaussian-Bernoulli restricted Boltzmann machines are generative models capable of learning an arbitrary probability distribution $p(\mathbf{x})$. Gaussian-Bernoulli RBMs and their derivatives (such as deep Boltzmann machines [Salakhutdinov and Hinton, 2009] and conditional RBMs [Taylor et al., 2006]) have found applications in image generation, collaborative filtering, and time series forecasting. Though RBMs have fallen out of fashion in the wake of state-of-the-art generative models such as variational autoencoders and generative adversarial networks, they remain interesting to study because they can encode continuous-valued data into stochastic binary data, can learn complex multimodal real-word distributions, and can be easily sampled from conditionally. Though Gaussian-Bernoulli RBMs can theoretically learn any smooth probability density [Gu et al., 2022], they are notoriously difficult to train, leading many researchers to propose new sampling methods in an effort to improve training robustness [Liao et al., 2022]. In this work, we expand upon the previous work of Fisher et al. [2018], which attempts to improve training by introducing an adversarial component to standard RBM contrastive divergence learning.

1.2 Aims

1. Through our review of the RBM literature, we discovered that there is no overview of Gaussian-Bernoulli RBMs that describes the architecture and training procedure in detail and includes relevant proofs and derivations. Our first aim is to present such an overview. In Section 2, we review Gaussian-Bernoulli RBMs in detail, covering their architecture, the Gibbs sampling procedure, and the contrastive divergence loss.
2. The adversarial training framework proposed by Fisher et al. [2018] for RBMs with arbitrarily defined energy functions is exciting, but the work itself lacks detailed explana-

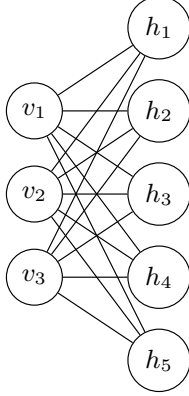


Figure 1: Example RBM architecture with 3 visible nodes and 5 hidden nodes. Bidirectional weights (representing $\mathbf{W}$) connect the visible and hidden layers. The biases μ and $\mathbf{b}$ on the visible and hidden layers respectively are not pictured.

tions and a publicly available implementation. Our second aim is to summarize their framework and provide a working implementation of the adversarial training algorithm applied to Gaussian-Bernoulli RBMs. We also experiment with more complex discriminators, treating the RBM as we would a generator in a generative adversarial network. We review the proposed framework in Section 3 and report empirical results in Section 4; we provide an implementation of adversarially trained Gaussian-Bernoulli RBMs at <https://github.com/ajy25/adversarialRBMs>.

3. Conditional RBMs transform generic RBMs into conditional generative models, but as derivatives of Gaussian-Bernoulli RBMs themselves, they may suffer from the same training difficulties that affect RBMs. Our final aim is to present a flexible framework for defining conditional RBMs and propose a modified adversarial training algorithm to work in the conditional case. This is done in Section 3, and empirical results are reported in in Section 4.

2 In-depth overview of Gaussian-Bernoulli RBMs

2.1 Architecture

A Boltzmann machine is a Markov random field with edges between all pairs of nodes. Some nodes are designated as *hidden* nodes that add to the expressivity of the Markov random field, while the remaining nodes, called *visible* nodes, together represent elements in feature space. A restricted Boltzmann machine is a Boltzmann machine that satisfies an additional constraint: there cannot be edges between any two visible nodes or any two hidden nodes. This constraint effectively transforms the Boltzmann machine into a two-layered stochastic neural network, as illustrated in Figure 1.

We denote the visible layer as $\mathbf{v}$ and the hidden layer as $\mathbf{h}$. Formally, an RBM is defined by an energy function $E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})$ that maps the visible and hidden values to a scalar, dependent on the weights between nodes. The joint probability density function over the hidden and visible nodes may then be defined as follows:

$$p(\mathbf{v}, \mathbf{h}; \boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} e^{-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})}. \quad (1)$$

Z is the partition function which in effect normalizes $\exp(-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))$ to satisfy the characteristics of a probability density function; Z is computationally intractable to compute. Hence, our training scheme utilizes the energy function exclusively, and we note that

$$p(\mathbf{v}, \mathbf{h}) \propto \exp(-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})).$$

In this work, we use the parameterization of the Gaussian-Bernoulli RBM originally defined by Cho et al. [2011]:

$$\begin{aligned} E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) &= \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} - \sum_j h_j b_j - \sum_{i,j} \frac{v_i W_{ij} h_j}{\sigma_i^2} \\ &= \frac{1}{2} \left(\frac{\mathbf{v} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \right)^\top \left(\frac{\mathbf{v} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \right) - \mathbf{h}^\top \mathbf{b} - \left(\frac{\mathbf{v}}{\boldsymbol{\sigma}^2} \right)^\top \mathbf{W} \mathbf{h}. \end{aligned} \quad (2)$$

The parameters of an RBM with n_{vis} visible nodes and n_{hid} hidden nodes, $\boldsymbol{\theta}$, include: (a) the weights matrix $\mathbf{W} \in \mathbb{R}^{n_{\text{vis}} \times n_{\text{hid}}}$, which represents the edges between the visible and hidden units, (b) the bias on the hidden and visible nodes, $\mathbf{b} \in \mathbb{R}^{n_{\text{hid}}}$ and $\boldsymbol{\mu} \in \mathbb{R}^{n_{\text{vis}}}$, respectively, and (c) the variance $\boldsymbol{\sigma}^2 \in \mathbb{R}^{n_{\text{vis}}}$. From Equation 2, we can derive the following conditional probabilities, where $\mathcal{N}(\cdot; \mu, \sigma^2)$ denotes the Gaussian probability density function with expectation μ and variance σ^2 , and $\text{MVN}(\cdot; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ denotes the multivariate Gaussian probability density function with expectation $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$:

$$\begin{aligned} p(\mathbf{v}|\mathbf{h}; \boldsymbol{\theta}) &= \prod_i \left(\mathcal{N} \left(v_i; \mu_i + \sigma_i \sum_j W_{ij} h_j, \sigma_i^2 \right) \right) \\ &= \text{MVN}(\mathbf{v}; \mathbf{W} \mathbf{h} + \boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)), \end{aligned} \quad (3)$$

$$p(\mathbf{h} = \mathbf{1}|\mathbf{v}; \boldsymbol{\theta}) = \text{Sigmoid} \left(\mathbf{W}^\top \left(\frac{\mathbf{v}}{\boldsymbol{\sigma}^2} \right) + \mathbf{b} \right). \quad (4)$$

Above, the division sign denotes element-wise division. Note that Equation 4 gives a vector where the k -th element is the probability that the k -th hidden node has value 1. We include detailed derivations for these results in Appendix A1.

2.2 Sampling from the model distribution

Let p_θ denote the model distribution (i.e., the distribution learned by the RBM) and p_d denote the data distribution (i.e., the ground truth distribution from which the available data was sampled). For sake of notation, we use p_θ to denote both the joint distribution and marginal distribution depending on the surrounding context. Gibbs sampling [Geman and Geman, 1984] is a commonly used method for efficiently drawing samples from distributions represented by Markov random fields. In the case of restricted Boltzmann machines, visible nodes are conditionally independent of one another, as are hidden nodes:

$$\begin{aligned} p(v_1|v_2, \dots, v_{n_{\text{vis}}}, h_1, \dots, h_{n_{\text{hid}}}) &= p(v_1|h_1, \dots, h_{n_{\text{hid}}}) \perp\!\!\!\perp p(v_i|h_1, \dots, h_{n_{\text{hid}}}), \\ p(h_1|v_1, \dots, v_{n_{\text{vis}}}, h_2, \dots, h_{n_{\text{hid}}}) &= p(h_1|v_1, \dots, v_{n_{\text{vis}}}) \perp\!\!\!\perp p(h_j|v_1, \dots, v_{n_{\text{vis}}}). \end{aligned}$$

This property of restricted Boltzmann machines allows for “block” Gibbs sampling. Beginning with one sample from the standard multivariate Gaussian distribution, which we denote $\mathbf{v}_0$, we can sample $\hat{\mathbf{h}}_1$ from $p(\hat{\mathbf{h}}_1 = \mathbf{1}|\mathbf{v}_0; \boldsymbol{\theta})$. Then, we can sample $\hat{\mathbf{v}}_1$ from $p(\hat{\mathbf{v}}_1|\hat{\mathbf{h}}_1; \boldsymbol{\theta})$. This concludes one iteration of the Gibbs sampling algorithm. By repeating this process k times, we can approximately draw $\hat{\mathbf{v}}_k$ from p_θ . We provide the block Gibbs sampling algorithm for drawing samples from an RBM in Appendix A2.

2.3 Contrastive divergence learning, Kullback–Leibler divergence, and maximum likelihood

The Kullback–Leibler divergence (KL-divergence),

$$D_{\text{KL}}(P || Q) = \int P(x) \log \left(\frac{P(x)}{Q(x)} \right) dx,$$

is a measure of divergence between two probability distributions. Hinton [2002] introduced the minimization of the forward KL-divergence as an objective for training restricted Boltzmann machines. The measure is non-negative and zero if and only if the distributions are identical. The KL-divergence can be thought of simply as the information content of $P(x)$ minus the information content of $Q(x)$ in expectation over the distribution P ; it is thus a good measure of difference between probability

distributions. Unfortunately, we do not have access to the probability distributions for the model or the data, so the KL-divergence cannot be used as a loss function directly.

It turns out, however, that minimizing the *contrastive divergence* loss function is equivalent to minimizing the KL-divergence. We reformulate our training objective as follows: we aim to minimize the contrastive divergence loss.

$$\text{CD-Loss} = \langle E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} - \langle E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \quad (5)$$

Above, $\langle \cdot \rangle_{\mathbf{v}, \mathbf{h} \sim p}$ denotes the expectation over $\mathbf{v}$ and $\mathbf{h}$, where $\mathbf{v}$ is drawn from p and $\mathbf{h}$ is the RBM-learned encoding of $\mathbf{v}$.

The minimization of the forward KL-divergence is the same as the minimization of contrastive divergence loss:

$$\frac{\partial}{\partial \theta} D_{\text{KL}}(p_d \| p_\theta) \propto \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} - \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta}. \quad (6)$$

Further, minimizing the forward KL-divergence is the same as maximizing the log likelihood. Contrastive divergence learning is therefore an RBM-specific method of maximizing the model’s log likelihood of being in a state defined by $\mathbf{v}$ and $\mathbf{h}$. A thorough derivation of this key result is presented in Appendix A3. To provide some better intuition regarding the relationship between contrastive divergence and maximum likelihood, consider the laws of thermodynamics which—very simply put—suggest that higher energy states (i.e., high $E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})$) occur with lower probability; we aim to match the energy of model samples with the energy of data samples in order to learn the data distribution.

The gradients of the contrastive divergence loss with respect to the parameters are derived in Appendix A4 and are presented below.

$$\nabla_{\mathbf{W}} \text{CD-Loss} = \left\langle \frac{\mathbf{v}}{\boldsymbol{\sigma}^2} \mathbf{h}^\top \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} - \left\langle \frac{\mathbf{v}}{\boldsymbol{\sigma}^2} \mathbf{h}^\top \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta}, \quad (7)$$

$$\nabla_{\mathbf{b}} \text{CD-Loss} = \langle \mathbf{h} \rangle_{\mathbf{h} \sim p_d} - \langle \mathbf{h} \rangle_{\mathbf{h} \sim p_\theta}, \quad (8)$$

$$\nabla_{\boldsymbol{\mu}} \text{CD-Loss} = \left\langle \frac{\mathbf{v} - \boldsymbol{\mu}}{\boldsymbol{\sigma}^2} \right\rangle_{\mathbf{v} \sim p_d} - \left\langle \frac{\mathbf{v} - \boldsymbol{\mu}}{\boldsymbol{\sigma}^2} \right\rangle_{\mathbf{v} \sim p_\theta}, \quad (9)$$

$$\nabla_{\log(\boldsymbol{\sigma}^2)} \text{CD-Loss} = \left\langle \frac{\mathbf{v}}{\boldsymbol{\sigma}^2} \odot \mathbf{h} \mathbf{W}^\top - \frac{(\mathbf{v} - \boldsymbol{\mu})^2}{2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} - \left\langle \frac{\mathbf{v}}{\boldsymbol{\sigma}^2} \odot \mathbf{h} \mathbf{W}^\top - \frac{(\mathbf{v} - \boldsymbol{\mu})^2}{2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta}. \quad (10)$$

Above, $\odot$ indicates element-wise multiplication, and $(\cdot)^2$ indicates element-wise squaring. Note that, as consistent with the RBM literature, we learn the natural logarithm of the variance, $\log(\boldsymbol{\sigma}^2)$, rather than the variance, $\boldsymbol{\sigma}^2$. In practice, we utilize stochastic gradient descent to minimize the contrastive divergence loss (Equation 5). For each minibatch, the expectations $\langle \cdot \rangle$ are approximated as sample means across the minibatch.

3 Adversarial training

3.1 Asymmetry of the Kullback-Leibler divergence

The KL-divergence,

$$D_{\text{KL}}(P \| Q) = \int P(x) \log \left(\frac{P(x)}{Q(x)} \right) dx,$$

is weighted by the first distribution P . By maximizing the log likelihood (and thus the forward KL-divergence), we risk “oversmoothing” as visualized in Figure 2.

Unlike the forward KL-divergence, the reverse KL-divergence penalizes oversmoothing. Intuitively, to combat the oversmoothing problem, our training objective should combine the forward and reverse KL-divergences, such as the following, for arbitrary weights α and β :

$$\begin{aligned} \text{Loss} &= D_{\text{KL}}(p_\theta \| p_d) + \beta D_{\text{KL}}(p_d \| p_\theta) \\ &= \alpha \text{CD-Loss} + \beta D_{\text{KL}}(p_d \| p_\theta). \end{aligned} \quad (11)$$

However, we have no knowledge of p_d , which means we cannot easily estimate $D_{\text{KL}}(p_d \| p_\theta)$. Rather, we turn towards a new type of f -divergence defined by Fisher et al. [2018] that bounds $D_{\text{KL}}(p_d \| p_\theta)$.

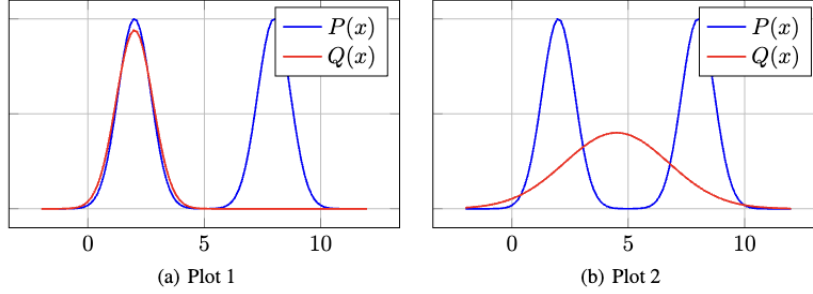


Figure 2: Illustrates the perils of forward KL-divergence $D_{\text{KL}}(P \parallel Q)$. In Plot 1, the interval $[5, 10]$ contributes significantly to $D_{\text{KL}}(P \parallel Q)$ since $D_{\text{KL}}(P \parallel Q)$ is weighted by $P(x)$. In Plot 2, $D_{\text{KL}}(P \parallel Q)$ is less than that of Plot 1 because the probability density is distributed more thoroughly over the support of $P(x)$, which demonstrates that optimizing for the forward KL-divergence could lead to “oversmoothing.”

3.2 An adversarial training framework

Fisher et al. [2018] introduce a *discriminator divergence* $D_{\text{Discriminator}}(P \parallel Q)$ in an effort to find an upper bound for the reverse KL-divergence $D_{\text{KL}}(Q \parallel P)$ and show that $D_{\text{Discriminator}}(P \parallel Q)$ closely mirrors $D_{\text{KL}}(Q \parallel P)$.

$$D_{\text{Discriminator}}(P \parallel Q) = - \int Q(x) \log \left(\frac{2P(x)}{P(x) + Q(x)} \right) dx \quad (12)$$

We verify that $\log(2) + D_{\text{Discriminator}}(P \parallel Q) \geq D_{\text{KL}}(Q \parallel P)$ in Appendix B1. In practice, we aim to compute $D_{\text{Discriminator}}(p_d \parallel p_\theta)$. We can interpret the value $p_d(\mathbf{v})/(p_d(\mathbf{v}) + p_\theta(\mathbf{v}))$ as the probability that a sample $\mathbf{v}$ was drawn from the data distribution. In a similar manner to generative adversarial networks, we estimate $p_d(\mathbf{v})/(p_d(\mathbf{v}) + p_\theta(\mathbf{v}))$ with a discriminator. The RBM acts as a generator by attempting to fool the discriminator, while the discriminator attempts to accurately distinguish between generated samples and real samples.

Fisher et al. [2018] use k nearest neighbor methods to estimate $p_d(\mathbf{v})/(p_d(\mathbf{v}) + p_\theta(\mathbf{v}))$. At each training step, we cache a minibatch sampled from the data distribution and a minibatch of the same size sampled from the model distribution. Then, for the i -th model sample from the subsequent minibatch, we find its k nearest neighbors among the cached samples. We then determine how many of the k neighbors originated from the model distribution, providing us with an estimate of $p_d(\mathbf{v})/(p_d(\mathbf{v}) + p_\theta(\mathbf{v}))$. Following Fisher et al. [2018], we define an adversarial loss as follows,

$$\text{Adversarial-Loss} = \langle -T(\mathbf{v}, \mathbf{h}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta}, \quad (13)$$

where $T(\mathbf{v}, \mathbf{h})$, called the *critic*, is designed to estimate $\log(2) + \log(p_d(\mathbf{v}, \mathbf{h})/(p_d(\mathbf{v}, \mathbf{h}) + p_\theta(\mathbf{v}, \mathbf{h})))$ —we allow T to be a function of both the visible and the encoded hidden layers to allow flexibility in defining T (Fisher et al. [2018], for example, actually use the hidden layer encodings of the visible layer). We show in Appendix B2 that the adversarial loss approximates the discriminator divergence. Fisher et al. [2018] prove that the gradient updates are simply

$$\frac{\partial}{\partial \theta} \text{Adversarial-Loss} = \text{Cov}_{\mathbf{v}, \mathbf{h} \sim p_\theta} \left[T(\mathbf{v}, \mathbf{h}), \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right], \quad (14)$$

which we estimate using the sample covariance computed from a minibatch. See Appendix B3 for the k nearest neighbors critic training algorithm. See Appendix B4 for the derivation for Equation 14.

3.3 Training classifiers for the critic function

We take inspiration from the training procedure of generative adversarial networks and propose the usage of more complex discriminators such as linear or deep classifiers that require supervised training for the prediction of $p_d(\mathbf{v})/(p_d(\mathbf{v}) + p_\theta(\mathbf{v}))$. We treat the RBM as a generator, and we train a discriminator to predict the probability that, given some $\mathbf{v}$, it was sampled from the data distribution rather than the model distribution. Then, our critic function is defined as

$$T(\mathbf{v}) = \log(2) + \log(\text{Discriminator}(\mathbf{v})), \quad (15)$$

where $\text{Discriminator}(\cdot)$ computes the probability of a sample originating from the data. We disregard $\mathbf{h}$ here because $\mathbf{h}$ would have been encoded by the model parameters.

A sensible complex discriminator on the visible layer would need to be trained on a set of “fake” (i.e., from the model distribution) and “real” (i.e., from the data distribution). Hence, we alternate between training the RBM for m epochs and training a classifier to predict $p_d(\mathbf{v})/(p_d(\mathbf{v}) + p_\theta(\mathbf{v}))$ for n epochs. In our work, we utilize a support vector classifier, though more experimentation could be done on deep classifiers.

3.4 A flexible conditional RBM framework

Taylor et al. [2006] transformed the RBM into a conditional generative model by incorporating dynamically changing biases as directed edges from a set of n_{cond} conditional nodes, which we will denote $\mathbf{u}$, to $\mathbf{b}$ and $\boldsymbol{\mu}$. We define the conditional RBM in a more flexible manner. We let the model parameters be functions of $\mathbf{u}$ rather than constants. In this work, we keep $\mathbf{W}$ constant and let

$$\begin{aligned}\boldsymbol{\mu} &: \mathbb{R}^{n_{\text{cond}}} \rightarrow \mathbb{R}^{n_{\text{vis}}}, \mathbf{u} \mapsto \text{Linear}(\mathbf{u}), \\ \mathbf{b} &: \mathbb{R}^{n_{\text{cond}}} \rightarrow \mathbb{R}^{n_{\text{hid}}}, \mathbf{u} \mapsto \text{Linear}(\mathbf{u}), \\ \log(\boldsymbol{\sigma}^2) &: \mathbb{R}^{n_{\text{cond}}} \rightarrow \mathbb{R}^{n_{\text{vis}}}, \mathbf{u} \mapsto \text{Linear}(\mathbf{u}).\end{aligned}$$

Above, $\text{Linear}(\cdot)$ denotes a linear map with appropriate input and output dimensions.

By learning $p(\mathbf{v}|\mathbf{u})$, we allow the conditional RBM to learn the variability of conditional distributions, avoiding the more complicated and thereby more difficult to learn joint distributions. Note that, assuming knowledge of $p(\mathbf{u})$, we have access to $p(\mathbf{v}|\mathbf{u})p(\mathbf{u}) = p(\mathbf{v}, \mathbf{u})$, from which we could sample from marginal $p(\mathbf{v})$. Therefore, in cases where the distribution of $\mathbf{v}$ is complex but may be simplified by conditioning on $\mathbf{u}$, we can train a conditional RBM rather than a generic RBM to improve learning outcomes.

3.5 Adversarial training of conditional RBMs

The oversmoothing issue with maximum likelihood can also occur when training conditional RBMs. We wish to determine whether or not adversarial training can apply to conditional RBMs as well. As described in Section 3.2, we wish to define a critic function $T(\mathbf{v}, \mathbf{h}, \mathbf{u})$ that estimates $\log(2) + \log(p_d(\mathbf{v}, \mathbf{h}|\mathbf{u})/(p_d(\mathbf{v}, \mathbf{h}|\mathbf{u}) + p_\theta(\mathbf{v}, \mathbf{h}|\mathbf{u})))$. In particular, we treat $\mathbf{h}$ as a latent space encoding of $\mathbf{v}$ and attempt to estimate $T(\mathbf{h}, \mathbf{u})$, marginalizing over $\mathbf{v}$.

In our conditional RBM framework, we allow the parameters $\boldsymbol{\mu}, \mathbf{b}, \boldsymbol{\sigma}^2$ to be arbitrarily defined functions of $\mathbf{u}$, leaving the visible-hidden connections $\mathbf{W}$ independent of the conditional values. The intuition behind leaving $\mathbf{W}$ independent of $\mathbf{u}$ is simple: though we wish to learn $p(\mathbf{v}|\mathbf{u})$, we let the model learn relationships between different conditions through $\mathbf{W}$ so that each $\mathbf{v}$ in feature space will be encoded into latent space in a somewhat $\mathbf{u}$ -independent manner. To illustrate this idea, suppose $\mathbf{u}$ is a single binary variable. If we let all parameters $\boldsymbol{\theta}$ be functions of $\mathbf{u}$, the connections between and biases on the visible and hidden units would be unique for either value of $\mathbf{u}$, which leaves the encoding of drastically different visible layer values into similar hidden layer values a possibility.

To provide the critic with the conditional information required to approximate $p_d(\mathbf{h}|\mathbf{u})/(p_d(\mathbf{h}|\mathbf{u}) + p_\theta(\mathbf{h}|\mathbf{u}))$, we work with $\mathbf{h}$ concatenated with $n_{\text{hid}} \cdot \hat{\mathbf{u}}$ in both our caching step and critic probability prediction step, where $\hat{\mathbf{u}}$ denotes the unit vector of $\mathbf{u}$. Recall that $\mathbf{h}$ takes on values of 1s and 0s; if we wish to work with distance-dependent discriminators, we should scale $\hat{\mathbf{u}}$ appropriately so that the conditional values are given enough weight in comparison to the hidden layer encodings.

We train the conditional RBM following Algorithm 4 in Appendix Appendix B5 and only apply the adversarial loss gradient update to $\mathbf{W}$.

4 Experiments

4.1 Unsupervised learning of Gaussian mixtures

We conducted experiments on two dimensional mixtures of Gaussians. We trained an RBM with 2 visible units and 20 hidden units on a 5×5 grid of Gaussians, with 1 Gibbs sampling step, learning

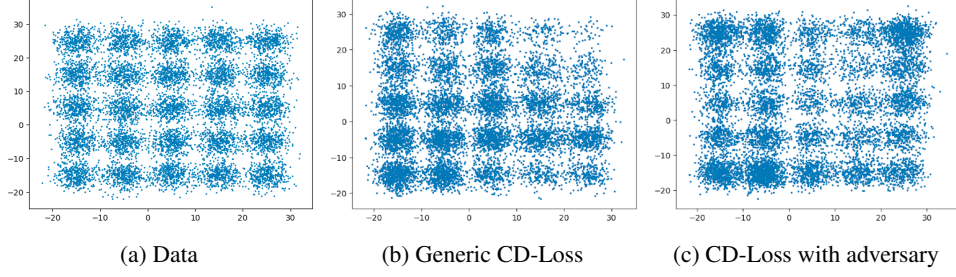


Figure 3: Gaussian-Bernoulli RBM training on a mixture of Gaussians. Plot (a) depicts 10000 samples drawn from a 5×5 grid of Gaussians. Plots (b) and (c) depict 10000 samples drawn from the RBMs. In Plot (c), the RBM was trained following the adversarial training procedure with the k nearest neighbors critic described in Section 3.2.

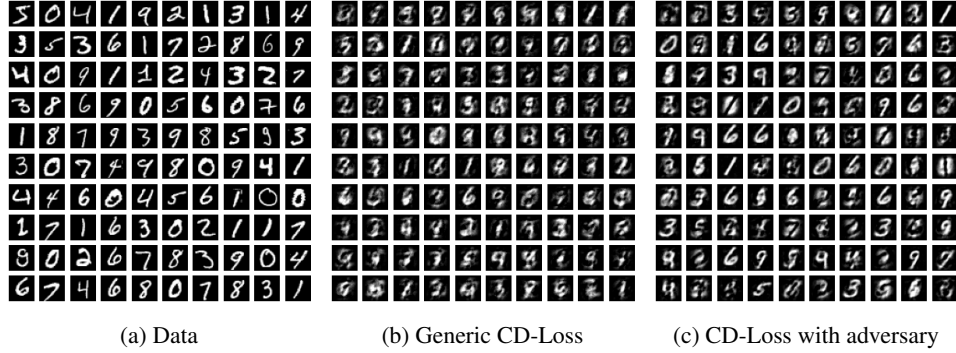


Figure 4: RBM training on MNIST. Plot (a) depicts 100 samples from the data distribution. Plots (b) and (c) depict 100 samples drawn from the trained RBM. In Plot (c), the RBM was trained following the adversarial training procedure with the support vector classifier discriminator described in Section 3.4.

rate set to 0.01, $\gamma = 0.1$, γ -delay = 10, batches of size 100, for a total of 400 epochs. Adversarial training was done following Section 3.2. The empirical results are presented in Figure 3.

4.2 Unsupervised learning of continuous MNIST

We conducted experiments on the continuous MNIST dataset [LeCun et al., 2010] with the goal of testing the proposed support vector classifier discriminator described in Section 3.4. We utilize only the first 2000 training examples in this experiment due to the computational complexity of training the support vector classifier.

We allowed the RBMs 500 binary hidden nodes for learning MNIST. We trained the first RBM adversarially with batch size 100, with 20 Gibbs sampling steps, learning rate of 0.001, γ set to 0.9, for 50 epochs, of which the first 30 were trained with $\gamma = 1$. We re-trained the support vector classifier on every fifth epoch. The empirical results are shown in Figure 4. We note that our sampled handwritten digits were not as clear as those reported by Liao et al. [2022], who managed to improve RBM training by introducing an improved sampling method.

4.3 Learning continuous MNIST conditioned on labels

Conditional RBMs learn a distribution $p(\mathbf{x}|\mathbf{y})$. We let $\mathbf{x}$ denote the 28×28 pixel values from continuous MNIST, and we let $\mathbf{y}$ denote the corresponding one-hot encoded label. We can therefore train a conditional RBM with 784 visible nodes $\mathbf{v}$, 10 conditional nodes $\mathbf{u}$, and 300 binary hidden nodes $\mathbf{h}$ to learn $p(\mathbf{v}|\mathbf{u})$. In this experiment, we let the critic T be the k nearest neighbors critic described in Appendix B3. We utilized only the first 10000 training examples from the continuous

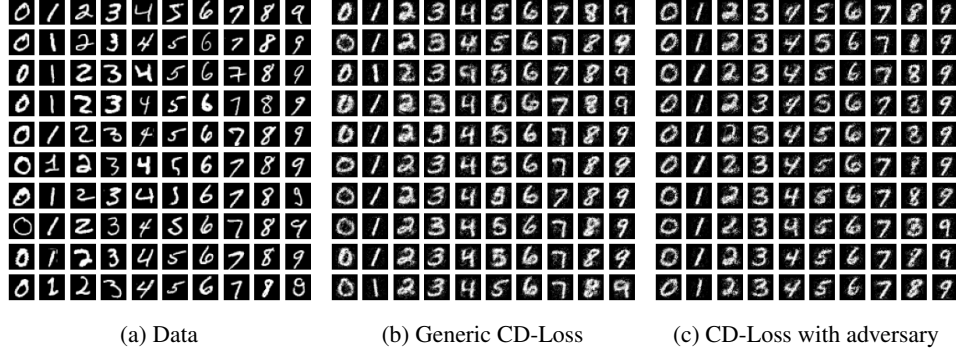


Figure 5: Conditional RBM training on MNIST. Plot (a) depicts 10 samples for each digit drawn from the data distribution. Plots (b) and (c) each depict 10 samples for each digit drawn from conditional RBMs trained to learn $p(\text{pixels}|\text{digit})$. In Plot (c), the conditional RBM was trained following the adversarial training procedure described in Section 3.5.

MNIST dataset in this experiment. We trained the first conditional RBM adversarially, with batches of size 100, 10 Gibbs sampling steps, learning rate of 0.001, and γ set to 0.9 for 45 epochs, of which the first 30 were trained with $\gamma = 1$. We trained another conditional RBM normally using the contrastive divergence loss with the same hyperparameters, with the exception of γ being set to 1. Figure 5 compares adversarial training with standard contrastive divergence training. We note that the model trained without the critic produced sharp, recognizable, and variable handwritten digits, as did the model trained with the critic; we cannot empirically observe any drastic differences between the two.

5 Discussion

Training Gaussian-Bernoulli RBMs is notoriously difficult due to their stochastic sampling methods. Over the past decade, researchers have proposed innovative sampling procedures in order to improve the training process. In this work, we investigated an adversarial training framework, which in theory should minimize the reverse KL-divergence in addition to forward KL-divergence despite relying on the simplest Gibbs sampling procedure for generating model samples.

One drawback of the discriminator divergence-based adversarial training framework proposed by Fisher et al. [2018] is that it is incompatible with automatic differentiation. Even with an ideal discriminator, the Gibbs sampling process of generating “fake” data from the model is stochastic, which prevents automatic differentiation from incorporating the model parameters into the computational graph. Note that, when training a generative adversarial network, the sampling process is deterministic after a probabilistic layer is initially sampled, allowing GANs to avoid this challenge. In this work, though we were able to extend the framework to more complex discriminators and conditional RBMs, we were unable to overcome the challenge of defining an automatic differentiation-compatible adversarial training algorithm for RBMs.

In addition, we achieved the following.

1. We provided detailed background for Gaussian-Bernoulli RBMs. First, we reviewed the key derivations related to the model architecture. Then, we illustrated the relationship between contrastive divergence loss, KL-divergence, and maximum likelihood. Finally, we filled in any gaps for the adversarial training framework proposed by Fisher et al. [2018].
2. We provided publicly-available implementations of RBMs and conditional RBMs. The implementations include the experimental training algorithms described in this work, such as the use of a support vector classifier as the discriminator and the adaption of the k nearest neighbors adversarial training method for conditional RBMs.

We remain optimistic that the robustness of Gaussian-Bernoulli RBM training can be improved. Future research directions could include the experimentation with deep classifiers or the development of conditional RBM specific discriminators.

References

- K. Cho, A. Ilin, and T. Raiko. Improved learning of gaussian-bernoulli restricted boltzmann machines. *Lecture Notes in Computer Science*, page 10–17, 2011. doi: 10.1007/978-3-642-21735-7_2.
- C. K. Fisher, A. M. Smith, and J. R. Walsh. Boltzmann encoded adversarial machines, 2018.
- S. Geman and D. Geman. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-6(6):721–741, 1984. doi: 10.1109/TPAMI.1984.4767596.
- L. Gu, L. Yang, and F. Zhou. Approximation properties of gaussian-binary restricted boltzmann machines and gaussian-binary deep belief networks. *Neural Netw*, 153:49–63, June 2022.
- G. E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002. doi: 10.1162/089976602760128018.
- Y. LeCun, C. Cortes, and C. Burges. Mnist handwritten digit database. *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, 2, 2010.
- R. Liao, S. Kornblith, M. Ren, D. J. Fleet, and G. Hinton. Gaussian-bernoulli rbms without tears, 2022.
- R. Salakhutdinov and G. Hinton. Deep boltzmann machines. In D. van Dyk and M. Welling, editors, *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, volume 5 of *Proceedings of Machine Learning Research*, pages 448–455, Hilton Clearwater Beach Resort, Clearwater Beach, Florida USA, 16–18 Apr 2009. PMLR. URL <https://proceedings.mlr.press/v5/salakhutdinov09a.html>.
- G. W. Taylor, G. E. Hinton, and S. Roweis. Modeling human motion using binary latent variables. In B. Schölkopf, J. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems*, volume 19. MIT Press, 2006. URL https://proceedings.neurips.cc/paper_files/paper/2006/file/1091660f3dff84fd648efe31391c5524-Paper.pdf.

Appendix

A1: Conditional probability derivations

To keep things compact, we will liberally use integral notation so that I can "integrate" over a vector. We denote $d\mathbf{v}$ as integration over all dimensions of $\mathbf{v}$.

$$\begin{aligned}
p_\theta(\mathbf{v}|\mathbf{h}) &= \frac{p_\theta(\mathbf{v}, \mathbf{h})}{\int p_\theta(\mathbf{v}, \mathbf{h}) d\mathbf{v}} \\
&= \frac{\frac{1}{Z(\boldsymbol{\theta})} e^{-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})}}{\int \frac{1}{Z(\boldsymbol{\theta})} e^{-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})} d\mathbf{v}} \\
&= \frac{\exp(-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}))}{\int \exp(-E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})) d\mathbf{v}} \\
&= \frac{\exp(\sum_j h_j b_j + \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2})}{\int \exp(\sum_j h_j b_j + \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2}) d\mathbf{v}} \\
&= \frac{\exp(\sum_j h_j b_j) \exp(\sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2})}{\exp(\sum_j h_j b_j) \int \exp(\sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2}) d\mathbf{v}} \\
&= \frac{\exp(\sum_i \sum_j \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2})}{\int \exp(\sum_i \sum_j \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2}) d\mathbf{v}} \\
&= \frac{\prod_i \exp(\sum_j \frac{v_i W_{i,j} h_j}{\sigma_i} - \frac{(v_i - \mu_i)^2}{2\sigma_i^2})}{\int \prod_i \exp(\sum_j \frac{v_i W_{i,j} h_j}{\sigma_i} - \frac{(v_i - \mu_i)^2}{2\sigma_i^2}) d\mathbf{v}} \\
&= \frac{\prod_i \int \exp(\sum_j \frac{v_i W_{i,j} h_j}{\sigma_i} - \frac{(v_i - \mu_i)^2}{2\sigma_i^2}) dv_i}{\prod_i \int \exp(\sum_j \frac{v_i W_{i,j} h_j}{\sigma_i} - \frac{(v_i - \mu_i)^2}{2\sigma_i^2}) dv_i} \\
&= \frac{\prod_i \int \exp(-\frac{1}{2\sigma_i^2} v_i^2 + (\frac{\mu_i}{\sigma_i^2} + \sum_j \frac{W_{i,j} h_j}{\sigma_i}) v_i - \frac{\mu_i^2}{2\sigma_i^2}) dv_i}{\prod_i \int \exp(-\frac{1}{2\sigma_i^2} v_i^2 + (\frac{\mu_i}{\sigma_i^2} + \sum_j \frac{W_{i,j} h_j}{\sigma_i}) v_i - \frac{\mu_i^2}{2\sigma_i^2}) dv_i} \\
&\stackrel{n}{=} \frac{\prod_i \sqrt{\frac{\pi}{\frac{1}{2\sigma_i^2}}} \exp(\frac{(\frac{\mu_i}{\sigma_i^2} + \sum_j \frac{W_{i,j} h_j}{\sigma_i})^2}{4(\frac{1}{2\sigma_i^2})} - \frac{\mu_i^2}{2\sigma_i^2})}{\prod_i \sigma_i \sqrt{2\pi} \exp(\frac{1}{2}(\frac{\mu_i}{\sigma_i} + \sum_j W_{i,j} h_j)^2 - \frac{\mu_i^2}{2\sigma_i^2})} \\
&= \frac{\prod_i \exp(\sum_j \frac{v_i W_{i,j} h_j}{\sigma_i} - \frac{(v_i - \mu_i)^2}{2\sigma_i^2})}{\prod_i \sigma_i \sqrt{2\pi} \exp(\frac{1}{2}(\frac{\mu_i}{\sigma_i} + \sum_j W_{i,j} h_j)^2 - \frac{\mu_i^2}{2\sigma_i^2})}
\end{aligned}$$

For step n , we use the generalized Gaussian integral

$$\int_{-\infty}^{\infty} e^{-ax^2 - bx - c} = \sqrt{\frac{\pi}{a}} e^{\frac{b^2}{4a} - c}.$$

We now have a product over i in both the numerator and the denominator, and so we can move the exponents around, as seen by the following.

$$\begin{aligned}
&= \prod_i \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left(\sum_j \frac{v_i W_{i,j} h_j}{\sigma_i} - \frac{(v_i - \mu_i)^2}{2\sigma_i^2} - \frac{1}{2}\left(\frac{\mu_i}{\sigma_i} + \sum_j W_{i,j} h_j\right)^2 + \frac{\mu_i^2}{2\sigma_i^2}\right) \\
&= \prod_i \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left(\frac{v_i}{\sigma_i} \sum_j W_{i,j} h_j - \frac{(v_i - \mu_i)^2}{2\sigma_i^2} - \frac{1}{2}\left(\sum_j W_{i,j} h_j\right)^2 - \frac{\mu_i}{\sigma_i} \sum_j W_{i,j} h_j\right) \\
&= \prod_i \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left(\frac{1}{-2\sigma_i^2}(-2\sigma_i v_i \sum_j W_{i,j} h_j + (v_i - \mu_i)^2 + \sigma_i^2 (\sum_j W_{i,j} h_j)^2 - 2\sigma_i \mu_i \sum_j W_{i,j} h_j)\right) \\
&= \prod_i \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left(\frac{1}{-2\sigma_i^2}((v_i - \mu_i)^2 - 2\sigma_i (\sum_j W_{i,j} h_j)(\mu_i + v_i) + \sigma_i^2 (\sum_j W_{i,j} h_j)^2)\right) \\
&= \prod_i \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left(\frac{1}{-2\sigma_i^2}((v_i - \mu_i)^2 - 2\sigma_i (\sum_j W_{i,j} h_j)(\mu_i + v_i) + \sigma_i^2 (\sum_j W_{i,j} h_j)^2)\right) \\
&= \prod_i \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left(\frac{1}{-2\sigma_i^2}(((v_i - \mu_i) - \sigma_i (\sum_j W_{i,j} h_j))^2)\right) \\
&= \prod_i \mathcal{N}\left(v_i; \mu_i + \sigma_i (\sum_j W_{i,j} h_j), \sigma_i\right)
\end{aligned}$$

We have the correct derivation for the conditional probabilities for $\mathbf{v}$ given $\mathbf{h}$. Now for the other direction. We will present the derivation for the k th element of $\mathbf{h}$. Note that since the hidden layer is Bernoulli, its state space is discrete. Thus, we have the following.

$$\begin{aligned}
p_\theta(h_k = 1 \mid \mathbf{v}) &= \frac{p_\theta(\mathbf{v}, \mathbf{h}, h_k = 1)}{p_\theta(\mathbf{v}, \mathbf{h}, h_k = 1) + p_\theta(\mathbf{v}, \mathbf{h}, h_k = 0)} \\
&= \frac{1}{1 + \frac{p_\theta(\mathbf{v}, \mathbf{h}, h_k = 0)}{p_\theta(\mathbf{v}, \mathbf{h}, h_k = 1)}} \\
&= \frac{1}{1 + \frac{\exp(-E(\mathbf{v}, \mathbf{h}, h_k = 0))}{\exp(-E(\mathbf{v}, \mathbf{h}, h_k = 1))}} \\
&= \frac{1}{1 + \frac{\exp\left(\sum_{j \neq k} h_j b_j + \sum_{i, j \neq k} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2}\right)}{\exp\left(\sum_{j \neq k} h_j b_j + \sum_{i, j \neq k} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} + b_k + \sum_i \frac{v_i W_{i,j}}{\sigma_i}\right)}} \\
&= \frac{1}{1 + \exp\left(-\left(b_k + \sum_i \frac{v_i W_{i,j}}{\sigma_i}\right)\right)} \\
&= \text{Sigmoid}\left(b_k + \sum_i \frac{v_i W_{i,j}}{\sigma_i}\right)
\end{aligned}$$

Step 1 works because the top and the bottom expression are identical except the terms left in the next step. Thus, we have the correct derivation of $p_\theta(\mathbf{h} \mid \mathbf{v})$.

A2: Block Gibbs sampling algorithm

Algorithm 1 is a simple block Gibbs sampling algorithm we use to sample from an RBM. We do not use any other tricks such as temperature driven sampling or persistent contrastive divergence. The output, $\mathbf{v}_{\text{current}}$, represents an approximate sample from the model distribution p_θ .

Sometimes we do not wish to begin with standard normal initialization. For example, if we wish to encode a given $\mathbf{v}$ into $\mathbf{h}$, or if we wish to impute missing values of $\mathbf{v}$, we would want to do without the random initialization step. Algorithm 2 takes care of this case.

Algorithm 1 Block Gibbs sampling with standard normal initialization, n -gibbs steps

```
1: procedure BLOCKGIBBSFROMRANDN( $n$ -gibbs)
2:   Sample  $v_0$  from MVN ( $\mathbf{0}$ ,  $\text{diag}(\mathbf{1})$ )
3:    $v_{\text{current}} \leftarrow v_0$ 
4:   for  $i$  in  $1 : n$ -gibbs do
5:     Sample  $h_{\text{current}}$  from Sigmoid ( $\mathbf{W}^\top (\frac{v_{\text{current}}}{\sigma^2}) + \mathbf{b}$ )
6:     Sample  $v_{\text{current}}$  from MVN ( $\mathbf{W}h_{\text{current}} + \boldsymbol{\mu}$ ,  $\text{diag}(\sigma^2)$ )
7:   end for
8:   return  $v_{\text{current}}, h_{\text{current}}$ 
9: end procedure
```

Algorithm 2 Block Gibbs sampling without standard normal initialization, n -gibbs steps

```
1: procedure BLOCKGIBBS( $v_{\text{data}}, n$ -gibbs)
2:    $v_{\text{current}} \leftarrow v_{\text{data}}$ 
3:   for  $i$  in  $1 : n$ -gibbs do
4:     Sample  $h_{\text{current}}$  from Sigmoid ( $\mathbf{W}^\top (\frac{v_{\text{current}}}{\sigma^2}) + \mathbf{b}$ )
5:     Sample  $v_{\text{current}}$  from MVN ( $\mathbf{W}h_{\text{current}} + \boldsymbol{\mu}$ ,  $\text{diag}(\sigma^2)$ )
6:   end for
7:   return  $v_{\text{current}}, h_{\text{current}}$ 
8: end procedure
```

A3: Minimizing contrastive divergence loss is equivalent to maximizing the log likelihood

We first show that minimizing the forward KL divergence is equivalent to maximizing the log likelihood of the data. Let $\mathbf{V} \sim p_d(\mathbf{v})$ denote the dataset the RBM is being trained on. Next we denote $\hat{p}_\theta(\mathbf{v}) = \sum_{\mathbf{h}} p_\theta(\mathbf{v}, \mathbf{h})$, the marginal probability (summed over all possible hidden unit configurations, which is discrete since they are Bernoulli). Notice that this is a probability distribution:

$$\int \hat{p}_\theta(\mathbf{v}) d\mathbf{v} = \int \sum_{\mathbf{h}} p_\theta(\mathbf{v}, \mathbf{h}) d\mathbf{v} = 1.$$

Since we are trying to show that minimizing the forward KL-divergence is the same as maximizing the log likelihood of the data, we will compute the KL-divergence over the dataset. In the following proof, we have annotated lines that deserve a more thorough explanation.

$$\begin{aligned} D_{\text{KL}}(p_d || p_\theta) &\stackrel{1}{=} \sum_{\mathbf{v} \in \mathbf{V}} p_d(\mathbf{v}) \log \left(\frac{p_d(\mathbf{v})}{\hat{p}_\theta(\mathbf{v})} \right) \\ &= \sum_{\mathbf{v} \in \mathbf{V}} p_d(\mathbf{v}) (\log(p_d(\mathbf{v})) - \log(\hat{p}_\theta(\mathbf{v}))) \\ &\stackrel{2}{=} - \sum_{\mathbf{v} \in \mathbf{V}} p_d(\mathbf{v}) \log(\hat{p}_\theta(\mathbf{v})) \\ &\stackrel{3}{\propto} - \sum_{\mathbf{v} \in \mathbf{V}} \log \hat{p}_\theta(\mathbf{v}) \\ &= -\log \prod_{\mathbf{v} \in \mathbf{V}} \hat{p}_\theta(\mathbf{v}). \end{aligned}$$

Below, we offer more details on the derivation above.

1. We apply the definition of D_{KL} . We are working in the discrete case (given a data set $\mathbf{V} = \{v_i\}_i$, so we replace the integral with a summation.
2. Note that p_d does not depend on θ , so the partial derivative is 0.
3. Because $p_d(\mathbf{v})$ is always positive, minimizing the previous expression is equivalent to minimizing the current expression.

The final expression in the above derivation is the negative log likelihood of the data points according to the probability defined by our RBM.

Next, we wish to show that minimizing the negative log likelihood is equivalent to minimizing the contrastive divergence. Let $\mathbf{h} \in \mathbf{V}$ be sampled from the model's conditional distribution for $\mathbf{h}$ conditioned on $\mathbf{v}$. Again, we have included annotations for statements that deserve more thorough explanations.

$$\begin{aligned}
-\frac{\partial}{\partial \theta} \log \prod_{\mathbf{v} \in \mathbf{V}} \hat{p}_{\theta}(\mathbf{v}) &= -\frac{\partial}{\partial \theta} \sum_{\mathbf{v} \in \mathbf{V}} \log \hat{p}_{\theta}(\mathbf{v}) \\
&= -\frac{\partial}{\partial \theta} \sum_{\mathbf{v} \in \mathbf{V}} \log \sum_{\mathbf{h}} p_{\theta}(\mathbf{v}, \mathbf{h}) \\
&= -\frac{\partial}{\partial \theta} \sum_{\mathbf{v} \in \mathbf{V}} \log \sum_{\mathbf{h}} \frac{1}{Z(\theta)} e^{-E(\mathbf{v}, \mathbf{h}, \theta)} \\
&= -\frac{\partial}{\partial \theta} \sum_{\mathbf{v} \in \mathbf{V}} \log \frac{1}{Z(\theta)} + \log \sum_{\mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \theta)} \\
&= \sum_{\mathbf{v} \in \mathbf{V}} \frac{\partial}{\partial \theta} \log Z(\theta) - \sum_{\mathbf{v} \in \mathbf{V}} \frac{\partial}{\partial \theta} \log \sum_{\mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \theta)} \\
&= \sum_{\mathbf{v} \in \mathbf{V}} \frac{\partial}{\partial \theta} \log Z(\theta) - \sum_{\mathbf{v} \in \mathbf{V}} \frac{\frac{\partial}{\partial \theta} \sum_{\mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \theta)}}{\sum_{\mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \theta)}} \\
&= \sum_{\mathbf{v} \in \mathbf{V}} \frac{\partial}{\partial \theta} \log Z(\theta) + \sum_{\mathbf{v} \in \mathbf{V}} \sum_{\mathbf{h}} \frac{e^{-E(\mathbf{v}, \mathbf{h}, \theta)} \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta)}{\sum_{\mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h}, \theta)}} \\
&\stackrel{1}{=} |\mathbf{V}| \frac{\partial}{\partial \theta} \log Z(\theta) + \sum_{\mathbf{v} \in \mathbf{V}} \sum_{\mathbf{h}} p_{\theta}(\mathbf{v} | \mathbf{h}) \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \\
&\stackrel{2}{=} -|\mathbf{V}| \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} + |\mathbf{V}| \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&\stackrel{3}{\propto} -\left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} + \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d}.
\end{aligned}$$

Step 1 comes from the fact that

$$p_{\theta}(\mathbf{h} | \mathbf{v}) = \frac{p_{\theta}(\mathbf{v}, \mathbf{h})}{\hat{p}_{\theta}(\mathbf{v})} = \frac{\frac{1}{Z(\theta)} e^{-E(\mathbf{v}, \mathbf{h}, \theta)}}{\sum_{\mathbf{h}} \frac{1}{Z(\theta)} e^{-E(\mathbf{v}, \mathbf{h}, \theta)}}.$$

For 2, we expand on the work for the first term.

$$\begin{aligned}
\frac{\partial}{\partial \theta} \log Z(\theta) &= \frac{1}{Z(\theta)} \frac{\partial}{\partial \theta} Z(\theta) \\
&= \frac{1}{Z(\theta)} \frac{\partial}{\partial \theta} \sum_{\mathbf{h}} \int e^{-E(\mathbf{v}, \mathbf{h}, \theta)} d\mathbf{v} \\
&= \frac{1}{Z(\theta)} \sum_{\mathbf{h}} \int \frac{\partial}{\partial \theta} e^{-E(\mathbf{v}, \mathbf{h}, \theta)} d\mathbf{v} \\
&= \frac{1}{Z(\theta)} \sum_{\mathbf{h}} \int e^{-E(\mathbf{v}, \mathbf{h}, \theta)} \frac{\partial}{\partial \theta} \log e^{-E(\mathbf{v}, \mathbf{h}, \theta)} d\mathbf{v} \\
&= -\sum_{\mathbf{h}} \int p_{\theta}(\mathbf{v}, \mathbf{h}) \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) d\mathbf{v} \\
&= -\left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}}.
\end{aligned}$$

And we also provide an explanation for the second term. Since p_d does not have any $\mathbf{h}$ dependency, we have to use the model's conditional distribution to be able to take the expectation of the energy. Thus, that inner sum is actually just "sampling" an $\mathbf{h}$ from p_d .

Finally, for 3, we have to scale up the expectation by the size of the data set to equal the log likelihood of the entire dataset. It makes sense because if we had one extra dataset, we would essentially be adding another term to the log likelihood, so we do need to scale. The resulting expression is the contrastive divergence. Thus, we have proven that minimizing the forward KL-divergence is the equivalent to maximizing the log-likelihood and minimizing the contrastive divergence.

A4: Gradients of contrastive divergence loss with respect to parameters

First, we derive the gradients with respect to $\mathbf{W}$.

$$\begin{aligned}
\nabla_{\mathbf{W}} \text{CD-Loss} &= \nabla_{\mathbf{W}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\mathbf{W}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\mathbf{W}} \left\langle \sum_j h_j b_j + \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\mathbf{W}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\mathbf{W}} \left\langle \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\mathbf{W}} \left\langle \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \left\langle \frac{\mathbf{v}}{\boldsymbol{\sigma}^2} \mathbf{h}^{\top} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} - \left\langle \frac{\mathbf{v}}{\boldsymbol{\sigma}^2} \mathbf{h}^{\top} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}}
\end{aligned}$$

Next, we derive the gradients with respect to $\mathbf{b}$.

$$\begin{aligned}
\nabla_{\mathbf{b}} \text{CD-Loss} &= \nabla_{\mathbf{b}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\mathbf{b}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\mathbf{b}} \left\langle \sum_j h_j b_j + \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\mathbf{b}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\mathbf{b}} \left\langle \sum_j h_j \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\mathbf{b}} \left\langle \sum_j h_j \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \langle \mathbf{h} \rangle_{\mathbf{h} \sim p_d} - \langle \mathbf{h} \rangle_{\mathbf{h} \sim p_{\theta}}
\end{aligned}$$

Next, we derive the gradients with respect to $\boldsymbol{\mu}$.

$$\begin{aligned}
\nabla_{\boldsymbol{\mu}} \text{CD-Loss} &= \nabla_{\boldsymbol{\mu}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\boldsymbol{\mu}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\boldsymbol{\mu}} \left\langle \sum_j h_j b_j + \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\boldsymbol{\mu}} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\boldsymbol{\mu}} \left\langle -\sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_{\theta}} - \nabla_{\boldsymbol{\mu}} \left\langle -\sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \left\langle \frac{\mathbf{v} - \boldsymbol{\mu}}{\boldsymbol{\sigma}^2} \right\rangle_{\mathbf{v} \sim p_d} - \left\langle \frac{\mathbf{v} - \boldsymbol{\mu}}{\boldsymbol{\sigma}^2} \right\rangle_{\mathbf{v} \sim p_{\theta}}
\end{aligned}$$

Finally, we derive the gradients with respect to $\log(\sigma^2)$.

$$\begin{aligned}
\nabla_{\log(\sigma^2)} \text{CD-Loss} &= \nabla_{\log(\sigma^2)} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} - \nabla_{\log(\sigma^2)} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\log(\sigma^2)} \left\langle \sum_j h_j b_j + \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} - \nabla_{\log(\sigma^2)} \langle -E(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\log(\sigma^2)} \left\langle \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \\
&\quad - \nabla_{\log(\sigma^2)} \left\langle \sum_{i,j} \frac{v_i W_{i,j} h_j}{\sigma_i} - \sum_i \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \nabla_{\log(\sigma^2)} \left\langle \sum_i \left(\frac{v_i}{\sigma_i} \left(\sum_j W_{i,j} h_j \right) - \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \\
&\quad - \nabla_{\log(\sigma^2)} \left\langle \sum_i \left(\frac{v_i}{\sigma_i} \left(\sum_j W_{i,j} h_j \right) - \frac{(v_i - \mu_i)^2}{2\sigma_i^2} \right) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} \\
&= \left\langle \frac{\mathbf{v}}{\sigma^2} \odot \mathbf{h} \mathbf{W}^\top - \frac{(\mathbf{v} - \boldsymbol{\mu})^2}{2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_d} - \left\langle \frac{\mathbf{v}}{\sigma^2} \odot \mathbf{h} \mathbf{W}^\top - \frac{(\mathbf{v} - \boldsymbol{\mu})^2}{2} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta}
\end{aligned}$$

B1: Discriminator divergence bounds reverse KL divergence

Let $\mathbf{s}$ denote all nodes of the Markov random field that is the RBM (i.e., let $\mathbf{s}$ be the concatenation of $\mathbf{v}$ and $\mathbf{h}$). Beginning with Equation 12,

$$\begin{aligned}
D_{\text{Discriminator}}(p_d \parallel p_\theta) &= - \int p_\theta(\mathbf{s}) \log \left(\frac{2p_d(\mathbf{s})}{p_d(\mathbf{s}) + p_\theta(\mathbf{s})} \right) d\mathbf{s} \\
&= -\log(2) - \int p_\theta(\mathbf{s}) \log \left(\frac{p_d(\mathbf{s})}{p_d(\mathbf{s}) + p_\theta(\mathbf{s})} \right) d\mathbf{s} \\
&= -\log(2) + \int p_\theta(\mathbf{s}) \log \left(1 + \frac{p_\theta(\mathbf{s})}{p_d(\mathbf{s})} \right) d\mathbf{s} \\
&\geq -\log(2) + \int p_\theta(\mathbf{s}) \log \left(\frac{p_\theta(\mathbf{s})}{p_d(\mathbf{s})} \right) d\mathbf{s} \\
&= -\log(2) + D_{\text{KL}}(p_\theta \parallel p_d),
\end{aligned}$$

$$D_{\text{Discriminator}}(p_d \parallel p_\theta) + \log(2) \geq D_{\text{KL}}(p_\theta \parallel p_d).$$

B2: Adversarial loss approximates discriminator divergence

Let $\mathbf{s}$ denote all nodes of the Markov random field that is the RBM (i.e., let $\mathbf{s}$ be the concatenation of $\mathbf{v}$ and $\mathbf{h}$). From Equation 12, we have

$$D_{\text{Discriminator}}(p_d \parallel p_\theta) = - \int p_\theta(\mathbf{s}) \log \left(\frac{2p_d(\mathbf{s})}{p_d(\mathbf{s}) + p_\theta(\mathbf{s})} \right) d\mathbf{s}.$$

From Equation 13, we have that

$$\begin{aligned}
\text{Adversarial-Loss} &= \langle -T(\mathbf{s}) \rangle_{\mathbf{s} \sim p_\theta} \\
&= - \int p_\theta(\mathbf{s}) T(\mathbf{s}) d\mathbf{s} \\
&\approx - \int p_\theta(\mathbf{s}) \left(\log(2) + \log \left(\frac{p_d(\mathbf{s})}{p_d(\mathbf{s}) + p_\theta(\mathbf{s})} \right) \right) d\mathbf{s} \\
&= - \int p_\theta(\mathbf{s}) \log \left(\frac{2p_d(\mathbf{s})}{p_d(\mathbf{s}) + p_\theta(\mathbf{s})} \right) d\mathbf{s}.
\end{aligned}$$

B3: RBM adversarial training procedure with k -NN critic

Fisher et al. [2018] provide a framework for adversarial training. We provide an explicit algorithm adapting their framework to Gaussian-Bernoulli RBM training, utilizing their proposed k nearest neighbors critic function on the hidden units to estimate $\log(2) + \log(p_d(\mathbf{v}, \mathbf{h}) / (p_d(\mathbf{v}, \mathbf{h}) + p_\theta(\mathbf{v}, \mathbf{h})))$. We aim to minimize

$$\text{Loss} = \gamma \text{CD-Loss} + (1 - \gamma) \text{Adversarial-Loss},$$

where

$$\text{Adversarial-Loss} = \langle -T(\mathbf{h}) \rangle_{\mathbf{h} \sim p_\theta}.$$

Suppose that we have cached a minibatch of m samples from the data distribution and m samples from the model distribution. Noting that a linear critic performs better in experiments, Fisher et al. [2018] define the critic function T as

$$T(\mathbf{h}) = \frac{2j}{k} - 1,$$

where for each new example from the model distribution, j denotes the number of its k nearest neighbors amongst the minibatch originating from the data distribution. We do not begin adversarial training until γ -delay epochs have passed so that model-sampled $\mathbf{h}$ more reasonably encode observed $\mathbf{v}$. We employ Algorithm 1 and Algorithm 2 to sample from the RBM, but we assume that they have been modified appropriately to work with batches; for ease of notation, we also let Algorithm 2 sample the same number of examples as Algorithm 1 during the sampling steps.

Algorithm 3 k -NN critic adversarial training

```

1: procedure TRAIN-KNN-CRITIC( $\mathbf{v}_{\text{train}}, n\text{-epochs}, n\text{-gibbs}, k, \gamma, \gamma\text{-delay}, \text{lr}$ )
2:   for  $i$  in  $1 : n_{\text{epochs}}$  do
3:     for  $\mathbf{v}_{\text{model}}$  in batch( $\mathbf{v}_{\text{train}}$ ) do
4:        $\_, \mathbf{h}_{\text{data}} \leftarrow \text{BlockGibbs}(\mathbf{v}_{\text{data}}, n\text{-gibbs})$ 
5:        $\mathbf{v}_{\text{model}}, \mathbf{h}_{\text{model}} \leftarrow \text{BlockGibbsFromRandN}(n\text{-gibbs})$ 
6:        $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \text{lr} \cdot \gamma \cdot \nabla_{\boldsymbol{\theta}} (\text{Mean}[E(\mathbf{v}_{\text{data}}, \mathbf{h}_{\text{data}})] - \text{Mean}[E(\mathbf{v}_{\text{model}}, \mathbf{h}_{\text{model}})])$ 
7:       if  $i \geq \gamma\text{-delay}$  and cache exists then:
8:          $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \text{lr}(1 - \gamma) \widehat{\text{Cov}}[T(\mathbf{h}_{\text{model}}; \text{cache}), \frac{\partial}{\partial \boldsymbol{\theta}} E(\mathbf{v}_{\text{model}}, \mathbf{h}_{\text{model}})]$ 
9:       end if
10:      cache  $\leftarrow (\mathbf{h}_{\text{data}}, \mathbf{h}_{\text{model}})$ 
11:    end for
12:  end for
13: end procedure

```

B4: Gradient of adversarial loss

Here we re-state the derivation originally given by Fisher et al. [2018].

$$\begin{aligned}
\frac{\partial}{\partial \theta} \text{Adversarial-Loss} &= \frac{\partial}{\partial \theta} \langle -T(\mathbf{v}, \mathbf{h}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \\
&= \frac{\partial}{\partial \theta} \sum_{\mathbf{h}} \int -T(\mathbf{v}, \mathbf{h}) p_\theta(\mathbf{v}, \mathbf{h}) d\mathbf{v} \\
&\stackrel{1}{=} \sum_{\mathbf{h}} \int -T(\mathbf{v}, \mathbf{h}) \frac{\partial}{\partial \theta} p_\theta(\mathbf{v}, \mathbf{h}) d\mathbf{v} \\
&= \sum_{\mathbf{h}} \int -T(\mathbf{v}, \mathbf{h}) p_\theta(\mathbf{v}, \mathbf{h}) \frac{\frac{\partial}{\partial \theta} p_\theta(\mathbf{v}, \mathbf{h})}{p_\theta(\mathbf{v}, \mathbf{h})} d\mathbf{v} \\
&= \sum_{\mathbf{h}} \int -T(\mathbf{v}, \mathbf{h}) p_\theta(\mathbf{v}, \mathbf{h}) \frac{\partial}{\partial \theta} \log p_\theta(\mathbf{v}, \mathbf{h}) d\mathbf{v} \\
&= \left\langle -T(\mathbf{v}, \mathbf{h}) \frac{\partial}{\partial \theta} \log p_\theta(\mathbf{v}, \mathbf{h}) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta}
\end{aligned}$$

Recall that $-\frac{\partial}{\partial \theta} \log p_\theta(\mathbf{v}, \mathbf{h}) = \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) - \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta}$. Thus,

$$\begin{aligned}
&\left\langle -T(\mathbf{v}, \mathbf{h}) \frac{\partial}{\partial \theta} \log p_\theta(\mathbf{v}, \mathbf{h}) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \\
&= \left\langle T(\mathbf{v}, \mathbf{h}) \left(\frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) - \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \right) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \\
&= \left\langle T(\mathbf{v}, \mathbf{h}) \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) - T(\mathbf{v}, \mathbf{h}) \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \\
&= \left\langle T(\mathbf{v}, \mathbf{h}) \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} - \langle T(\mathbf{v}, \mathbf{h}) \rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \left\langle \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}, \theta) \right\rangle_{\mathbf{v}, \mathbf{h} \sim p_\theta} \\
&= \text{Cov}_{\mathbf{v}, \mathbf{h} \sim p_\theta} \left[T(\mathbf{v}, \mathbf{h}), \frac{\partial}{\partial \theta} E(\mathbf{v}, \mathbf{h}) \right].
\end{aligned}$$

Step 1 is allowed since $T(\mathbf{v}, \mathbf{h})$ does not depend on the parameters of the model.

B5: Adversarial training algorithm

Algorithm 4 Adversarial training for conditional RBM

```

1: procedure TRAIN-CRITIC-CRBM( $\mathbf{v}_{\text{train}}, \mathbf{u}_{\text{train}}, n\text{-epochs}, n\text{-gibbs}, k, \gamma, \gamma\text{-delay}, \text{lr}$ )
2:   for  $i$  in  $1 : n\text{-epochs}$  do
3:     for  $\mathbf{v}_{\text{data}}, \mathbf{u}_{\text{data}}$  in batch( $\mathbf{v}_{\text{train}}, \mathbf{u}_{\text{train}}$ ) do
4:       Compute  $\theta$  from  $\mathbf{u}_{\text{data}}$  if necessary
5:        $\_, \mathbf{h}_{\text{data}} \leftarrow \text{BlockGibbs}(\mathbf{v}_{\text{data}}, n\text{-gibbs})$ 
6:        $\mathbf{v}_{\text{model}}, \mathbf{h}_{\text{model}} \leftarrow \text{BlockGibbsFromRandN}(n\text{-gibbs})$ 
7:        $\nabla \theta \leftarrow \cdot \nabla_\theta (\text{Mean}[E(\mathbf{v}_{\text{data}}, \mathbf{h}_{\text{data}})] - \text{Mean}[E(\mathbf{v}_{\text{model}}, \mathbf{h}_{\text{model}})])$ 
8:       if  $i \geq \gamma\text{-delay}$ , cache exists, and  $\theta = \mathbf{W}$  then:
9:          $\nabla \theta \leftarrow \gamma \cdot \nabla \theta + (1 - \gamma) \cdot \widehat{\text{Cov}}[T(\mathbf{h}_{\text{model}}, \mathbf{u}_{\text{data}}; \text{cache}), \frac{\partial}{\partial \theta} E(\mathbf{v}_{\text{model}}, \mathbf{h}_{\text{model}})]$ 
10:      end if
11:       $\theta \leftarrow \theta - \text{lr} \cdot \nabla \theta$ 
12:      cache  $\leftarrow (\text{concat}(\mathbf{h}_{\text{data}}, n_{\text{hid}} \cdot \hat{\mathbf{u}}_{\text{data}}), \text{concat}(\mathbf{h}_{\text{model}}, n_{\text{hid}} \cdot \hat{\mathbf{u}}_{\text{data}}))$ 
13:    end for
14:  end for
15: end procedure

```
